Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Desarrollo de Aplicaciones Web	Proyecto	**Desarrollo de Solución Web Full-Stack**
		Tutor: Ing. Victoria Castro

Objetivo:

Desarrollar una aplicación web completa y funcional que resuelva una problemática de gestión de información, aplicando una arquitectura desacoplada (Cliente-Servidor), persistencia robusta y despliegue orquestado en contenedores.

- **Organización del Trabajo**

El proyecto se realizará en equipos de 5 integrantes, estos equipos trabajaran de manera colaborativa durante todo el ciclo académico.

El tema es libre, sin embargo, se sugieren las siguientes líneas de trabajo.

Temáticas sugeridas:

1. **Gestión Hospitalaria:** Control de pacientes, citas y expedientes médicos.
2. **E-Commerce Académico:** Venta de libros, cursos o materiales universitarios.
3. **Sistema de Inventarios:** Control de stock, proveedores y alertas de reabastecimiento.

- **Requisitos Técnicos Obligatorios:**

Para que el proyecto sea evaluado, debe cumplir con los siguientes componentes tecnológicos:

1. **Backend:** Spring Boot 3.x (Java) utilizando arquitectura de N-Capas (Controladores, Servicios, Repositorios, Entidades) y patrón DTO.

2. **Base de Datos:** Persistencia en PostgreSQL con relaciones (1:N, N:M) mapeadas en JPA/Hibernate.

3. **API REST:** Implementación de verbos HTTP, códigos de estado y documentación con Swagger.

4. **Frontend**: SPA desarrollada en React con TypeScript, maquetación responsiva (Flexbox/Grid) y consumo de servicios con Axios/Fetch.

5. **Seguridad:** Control de acceso mediante JWT (JSON Web Tokens).

6. **Despliegue:** Uso de Docker (Dockerfiles) y Docker Compose para levantar todo el sistema (Frontend + Backend + DB) con un solo comando.

- **Guía de Entrega y Estructura del Repositorio**

A. Documento de Portada (PDF en Campus Virtual)

Se debe subir un archivo único por grupo que contenga:

- Nombres y apellidos completos.
- Número de carnet.
- Fecha de entrega.
- Enlace directo al repositorio (GitHub).

B. Estructura del Repositorio Git

El repositorio debe estar organizado de la siguiente manera:

```
/
├── backend/        # Código fuente de Spring Boot
├── frontend/       # Código fuente de React
├── database/       # Scripts de SQL o archivo del Diagrama Entidad-Relación (DER)
├── docker-compose.yml # Archivo de orquestación
└── README.md       # Manual y descripción
```

C. Contenido del Archivo README.md

Es obligatorio que el archivo principal del repositorio incluya:

1. **Descripción del Proyecto**: Qué problema resuelve y sus funciones principales.
2. **Diagrama Entidad-Relación:** Imagen o archivo del diseño de la base de datos.
3. **Manual de Despliegue:** Instrucciones paso a paso de cómo ejecutar el comando docker-compose up para levantar el sistema.
4. **Evidencias de Funcionamiento:**
    - Capturas de pantalla de la documentación en Swagger.
    - Tabla de rutas (Endpoints) del Backend.

• Entregable de la Defensa (Demostración)

El día de la defensa, el grupo deberá realizar una demostración en vivo que incluya:

1. **Ejecución en Docker:** Demostrar que el sistema levanta correctamente desde contenedores.
2. **Prueba de Seguridad:** Flujo completo de Login (Generación de token JWT).
3. **Consumo de API**: Mostrar cómo el Frontend solicita datos y el Backend responde.
4. **Interfaz de Usuario:** Navegación completa por las funcionalidades del sistema.

- **Rubrica de Evaluación**

Criterio de Evaluación	Excelente (5 pts)	Muy Bueno (4 pts)	Satisfactorio (3 pts)	En Proceso (2 pts)	Insuficiente (1 pt)
Despliegue con Docker	El sistema levanta con docker-compose up sin errores. Los 3 servicios (BD, BE, FE) se comunican perfectamente.	El sistema levanta, pero requirió un ajuste menor manual en el archivo de configuración durante la defensa.	Levantan los contenedores, pero hay errores de conexión entre el Frontend y el Backend.	Solo levanta un contenedor (ej. solo la DB o solo el Backend) de forma aislada.	No presenta archivos Docker o los contenedores no ejecutan.
Flujo de Seguridad (JWT)	Demuestra login exitoso, generación de token y acceso a rutas protegidas sin fallos.	El flujo JWT funciona, pero el token no se almacena o gestiona correctamente en el cliente (React).	Hay login, pero las rutas protegidas del backend no validan el token correctamente.	Solo existe el formulario de login visual, pero no hay validación real con el Backend.	No existe implementación de seguridad ni manejo de tokens.
Integración Front-Back	Consumo fluido de la API REST. Los datos se crean, listan y actualizan desde la interfaz React hacia Postgres.	El consumo funciona, pero presenta errores de renderizado o falta de tipado en algunas peticiones.	Solo funciona el consumo de datos (GET), pero falla el envío de información (POST/PUT).	La interfaz está desconectada del backend; usa datos estáticos (Mock data).	No hay conexión alguna entre el cliente y el servidor.
Documentación y Entrega	Portada completa con enlace al repo funcional y README con diagrama ER y manual de despliegue claro.	Entrega completa, pero el manual de despliegue es ambiguo o faltan capturas de Swagger.	Falta el diagrama ER o la definición de roles en la portada.	El enlace al repositorio es privado o la estructura de carpetas es desordenada.	No entregó portada ni documentación mínima en el repositorio.
Defensa Técnica	El equipo demuestra dominio total, explica la arquitectura y responde con solvencia todas las preguntas.	Explican bien el funcionamiento, pero titubean en preguntas sobre la lógica de los contenedores o JWT.	Solo un integrante habla; el resto muestra poco conocimiento del código integrado.	La demostración es accidentada y no logran explicar cómo se conectan las piezas.	El equipo no es capaz de explicar el funcionamiento del sistema presentado.